



Escuela
Politécnica
Superior

Formula 1 Winner Prediction: Combining Machine Learning and Deep Learning with Probability Calibration and Cloud Deployment



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:
Iván Álvarez García
Tutor/es:
Carlos Cano
Carlos Quesada

Mayo 2026



Universitat d'Alacant
Universidad de Alicante

Formula 1 Winner Prediction: Combining Machine Learning and Deep Learning with Probability Calibration and Cloud Deployment

Autor

Iván Álvarez García

Tutor/es

Carlos Cano

Ciencia de la Computación e Inteligencia Artificial

Carlos Quesada

Ciencia de la Computación e Inteligencia Artificial



Grado en Ingeniería Informática



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, Mayo 2026

Preamble

Formula 1 has become a highly data-driven environment where decisions are increasingly informed by predictive models. This study was motivated by the gap between the practical importance of pre-race forecasting and the limited methodological focus of many published F1 prediction systems, which often emphasise winner classification while giving little attention to probability reliability and operational deployment.

The subject of this thesis is pre-race winner prediction in Formula 1 using publicly available data. Its purpose is to design, evaluate, and deploy a reproducible end-to-end system that compares a classical tabular model (XGBoost) and a deep learning alternative (TabNet), and then calibrates their probabilistic outputs so that reported win probabilities better reflect observed frequencies.

The scope is restricted to information available before race start (qualifying, practice proxies, standings, weather, and circuit context), with evaluation focused on predictive performance and calibration under realistic season conditions. The work also includes a practical deployment pipeline using local containerisation and cloud services for race-weekend operation.

Acknowledgements

I would like to thank my supervisors, Carlos Cano and Carlos Quesada, for their guidance, technical feedback, and continuous support throughout this project. Their discussions were essential in shaping both the methodological decisions and the final structure of the work.

I also acknowledge the support provided by the Escuela Politecnica Superior of the Universidad de Alicante and the open-source communities behind FastF1, scikit-learn, XGBoost, and PyTorch, whose tools made this work technically feasible.

Finally, to all who believed in me even when I couldn't, and to Fernando Alonso, the driver who made me fall in love with this sport.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Context: data science and artificial intelligence in sport	1
1.3	Problem statement	2
1.4	Related work	2
1.5	System architecture	2
1.5.1	Why XGBoost on AWS Lambda	3
1.5.2	Why not TabNet on Lambda	3
1.5.3	Amazon S3 as the single source of truth	3
1.5.4	Amazon Athena for ad-hoc analysis	3
1.5.5	Google Sheets and Looker Studio for visualisation	4
1.5.6	Implementation tools	4
1.6	Model evaluation: beyond accuracy	4
1.7	Scope and limitations	5
2	Theoretical Background and State of the Art	7
2.1	Machine Learning in Sports Outcome Prediction	7
2.2	Formula 1 Race Outcome Prediction: Classical Machine Learning	7
2.3	Deep Learning for Tabular Racing Data	9
2.4	Race Simulation and Strategy: A Different Perspective	10
2.5	Open Problems	10
3	Objectives	13
4	Methodology	15
4.1	Data collection and preprocessing	15
4.1.1	Data sources	15
4.1.2	Temporal scope and train-test split	15
4.1.2.1	Empirical justification of the training window	15
4.1.3	Feature engineering	16
4.1.3.1	Qualifying performance	16
4.1.3.2	Recent form	16
4.1.3.3	Practice pace and aerodynamic context	16
4.1.3.4	Circuit-specific history	17
4.1.3.5	Constructor performance	17
4.1.3.6	Weather	18
4.1.3.7	Recency weighting	18
4.2	Model training	18
4.2.1	XGBoost configuration	18

4.2.2	TabNet configuration	18
4.3	Probability calibration	18
4.3.1	Temperature scaling	18
4.3.2	Model interpretability	19
4.4	Model evaluation	20
4.4.1	Metrics	20
4.4.2	Temporal validation	20
4.4.3	Probability snapshot artefacts	20
5	Development	23
5.1	System architecture and repository structure	23
5.2	Data ingestion and preprocessing pipeline	23
5.3	Feature engineering implementation	23
5.4	Model training pipeline	23
5.4.1	Shared training flow	23
5.4.2	Recency weighting in implementation	24
5.4.3	Probability calibration workflow	24
5.5	Inference and race-weekend operation	24
5.5.1	Dual-model prediction workflow	24
5.5.2	Full probability distributions	24
5.6	Interpretability artefacts in deployment	24
5.7	Cloud integration and observability	24
5.7.1	XGBoost deployment on AWS Lambda	25
5.7.2	S3 as central data and artefact bus	25
5.7.3	Athena layer for SQL analytics	25
5.7.4	Google Sheets synchronisation	25
5.7.5	Looker Studio integration	25
5.8	Reproducibility and engineering constraints	26
5.8.1	Reproducibility controls	26
5.8.2	Operational constraints observed	26
5.9	Development summary	26
6	Results and Discussion	27
6.1	Overall predictive accuracy	27
6.2	Season-by-season breakdown	27
6.2.1	2023: the Verstappen effect	28
6.2.2	2024: a genuinely competitive season	28
6.2.3	2025 and 2026: validation and test	28
6.3	Model agreement as an uncertainty signal	29
6.4	Probability calibration: R03 and R04 snapshots	29
6.5	Feature importance interpretation	31
6.6	Discussion	33
6.6.1	Why XGBoost outperforms TabNet	33
6.6.2	Sensitivity to competitive context	33
6.6.3	Limitations	33

7	Conclusions	35
	References	35
	List of Acronyms and Abbreviations	39

List of Figures

4.1	CFD velocity streamlines around an F1 car illustrating aerodynamic wake and downforce disturbance in dirty air. Image by Atif Masood (https://fetchcfid.com/view-project/1603), used with permission for non-commercial academic purposes.	17
4.2	Calibrated win-probability snapshots for 2026 R03 (Japan) and R04 (Miami), comparing XGBoost and TabNet over the top-10 drivers by mean probability in each round.	21
6.1	Looker Studio dashboard showing the four 2026 test-set predictions, both models, actual winners, and assigned win probabilities. XGBoost (<code>xgb_correct</code>) achieves 100%; TabNet (<code>tab_correct</code>) achieves 25%.	29
6.2	Win probability assigned to each model’s predicted driver per 2026 race. XGBoost (blue) is consistently more accurate; TabNet (orange) assigns very high probability to wrong predictions in R01 and R02.	30
6.3	Cumulative Brier Score (lower is better) over 2026 R01–R04 for XGBoost (blue) and TabNet (orange).	31
6.4	Cumulative positional MAE (lower is better) over 2026 R01–R04. XGBoost (blue) remains near zero throughout; TabNet (orange) starts high due to confident wrong predictions and converges after R03.	31
6.5	Feature importances for XGBoost (blue) and TabNet (orange) as displayed in the Looker Studio dashboard. XGBoost importance is dominated by <code>quali_gap_to_pole_s</code> ; TabNet distributes attention more evenly across qualifying, practice, and circuit features.	32

List of Tables

2.1	Comparison of representative F1 race outcome prediction works.	11
4.1	XGBoost winner-prediction accuracy on 2026 R1–R4 for two training windows.	16
4.2	Top features by cross-model agreement (combined XGBoost + TabNet ranking).	19
6.1	Winner-prediction accuracy by split and model.	27
6.2	Winner-prediction accuracy per season, both models.	28
6.3	Prediction outcomes conditioned on inter-model agreement.	29
6.4	Brier Scores for 2026 R03 and R04 probability snapshots (lower is better). . .	30
6.5	Feature importances from the last training run. XGBoost uses gain; TabNet uses attentive mask aggregation.	32

1 Introduction

1.1 Motivation

Formula 1 has always been, at its core, an engineering competition. What has changed over the last decade is where that engineering happens: increasingly, it happens in data. Modern teams process hundreds of gigabytes of telemetry per race weekend, run thousands of simulation laps overnight, and make split-second strategic decisions, tyre changes, fuel loads, pit windows, guided by probabilistic models rather than gut instinct. The sport has quietly become one of the most data-intensive environments in professional athletics.

This shift has not gone unnoticed outside the paddock. The intersection of motorsport and data science has attracted growing research interest, motivated both by the inherent prediction challenge, a race has twenty-two drivers, uncertain weather, and dozens of compounding variables, and by the practical value of accurate forecasting for broadcasters, fantasy leagues, and betting markets. Yet despite this interest, published work tends to rely on a single model and rarely addresses the reliability of the probability estimates it produces: knowing that a model assigns a 70% win probability to the pole-sitter is only meaningful if that figure is actually calibrated.

This project was born from that gap. The goal was not simply to build a race winner predictor, but to build one seriously: with two architecturally different models, with an honest evaluation of their probability calibration, and with a deployment setup that could be used in practice every race weekend.

1.2 Context: data science and artificial intelligence in sport

Sports analytics has evolved from simple statistics, win rates, point averages, into a fully-fledged discipline that combines large-scale data collection, feature engineering, and predictive modelling. Football clubs use expected goals models to evaluate transfers. Basketball teams rebuilt their entire offensive philosophies around shot-quality metrics. Baseball adopted sabermetrics decades ago and never looked back.

Formula 1 sits in a peculiar position within this landscape. On one hand, it generates richer data than almost any other sport: lap times accurate to the millisecond, tyre degradation curves, sector splits, weather readings taken trackside every few seconds. On the other hand, very little of that data is publicly available. Teams treat their telemetry as a competitive asset, and the official timing feed is deliberately limited. This makes F1 an interesting machine learning problem: the signal is there, but extracting it requires careful feature construction from the data that *is* accessible.

The FastF1 Python library (Schaffarczyk, 2025) changes that equation somewhat. It provides structured access to the official F1 timing API, exposing qualifying lap times, sector splits, practice session paces, weather readings, and race results going back several seasons.

It is what makes a project like this feasible at the university level: a reasonably rich feature set can be built without institutional access to proprietary telemetry.

1.3 Problem statement

The prediction target is binary: for each race, identify which driver will finish first. The input is entirely based on information available before the Sunday race start, primarily Saturday qualifying results, practice session paces, championship standings, and static circuit characteristics. This constraint is deliberate: a predictor that requires live race data is operationally useless.

The underlying difficulty is class imbalance. Out of twenty-two drivers on the grid, exactly one wins. A naive classifier that always predicts the pole-sitter would achieve roughly 40% accuracy on dry races and look competitive on paper, while producing predictions that have no probabilistic meaning whatsoever. Addressing this properly, through both modelling choices and post-hoc calibration, is central to this work.

A further challenge arises from the calendar itself. Sprint weekends, where a Sprint Race replaces FP2 and FP3 on Friday, alter the data available before Sunday’s race. The qualifying session still takes place on Saturday and remains the primary input, but practice pace proxies must be handled differently when FP2 and FP3 do not exist. The system addresses this with an explicit fallback mechanism described in Chapter 5.

1.4 Related work

Predicting Formula 1 race outcomes from qualifying and historical data has attracted growing research interest over the last decade. (Sicoie, 2022) explored ensemble classifiers on FastF1-derived features and found that grid position dominates short-term prediction but that historical driver performance adds consistent signal across a season. (O’Hanlon, 2023) combined qualifying pace with tyre strategy data and demonstrated that gradient boosting outperforms logistic regression on this task, reinforcing XGBoost as a strong baseline. More recently, (El Haber, Sawaya, & Attieh, 2025) and (Urdhwarshie, 2025) have examined neural approaches to motorsport prediction, though neither addresses probability calibration explicitly.

What distinguishes this project from prior work is the explicit treatment of probability calibration and the end-to-end deployment pipeline. Most academic papers evaluate models in retrospective cross-validation settings; here the system is live, making real predictions before each 2026 race and accumulating empirical evidence on calibration quality as the season progresses.

1.5 System architecture

The system is designed around a clear separation between the training environment and the inference environment. Training is computationally intensive and runs locally inside a Docker container, which ensures reproducibility regardless of the host machine. Inference, by

contrast, needs to be fast, cheap, and available after every qualifying session without manual intervention, which is where cloud deployment enters.

1.5.1 Why XGBoost on AWS Lambda

XGBoost was the natural candidate for serverless deployment. Its serialised model artefact is small (under 50 MB including encoders and the calibration temperature), it has no GPU dependency, and inference on a single race grid takes a few milliseconds. AWS Lambda covers all of this with zero infrastructure management: there is no server to maintain, no instance to keep running between race weekends, and costs are effectively negligible at this invocation frequency.

The Lambda function is invoked manually after qualifying, a deliberate choice given the low frequency of invocations (at most one per race weekend) and the fact that qualifying times are only definitive once all sessions have concluded and any penalties have been applied. Automating this step would add complexity with little practical benefit at this scale. The function is packaged as a Docker image stored in Amazon ECR, which avoids the package size limitations of ZIP-based deployments and makes the dependency environment completely deterministic.

1.5.2 Why not TabNet on Lambda

TabNet is built on PyTorch (Paszke et al., 2019), which adds roughly 1 GB of dependencies. That alone pushes the image beyond AWS Lambda’s practical limits and would incur significant cold-start latency. TabNet therefore runs as a local Docker service, invoked manually after qualifying via `docker compose`. This is an accepted trade-off: the model comparison the project aims for is preserved, and the limitation is documented rather than papered over.

1.5.3 Amazon S3 as the single source of truth

All persistent artefacts, trained models, probability calibration temperatures, prediction history, feature importance tables, and accuracy metrics, live in a single S3 bucket named `f1-winner-predictor-2026`. Using S3 as the central store rather than a database was a deliberate choice driven by simplicity: the data volume is tiny (a few hundred rows per season), the access pattern is sequential appends, and S3 is natively accessible from both Lambda and local Docker containers without any additional service. It also acts as a lightweight disaster recovery mechanism; if a local model file is corrupted or deleted, it can be restored from S3 without retraining.

Lambda and the local TabNet script both write to the same row in `predictions/history.csv`, keyed by year and round. A merge function ensures neither process overwrites the other’s columns, so the final record always contains both models’ predictions in a single, clean row.

1.5.4 Amazon Athena for ad-hoc analysis

Athena is configured to query the S3 CSV files using standard SQL. This was added not because it is strictly necessary for the weekly workflow, but because it makes exploratory analysis convenient without loading data locally. Any question like “what was XGBoost’s Brier Score in wet races only?” can be answered with a single SQL query against the S3 data,

without writing a Python script. Athena’s serverless, pay-per-query billing means there is no cost when it is not used.

1.5.5 Google Sheets and Looker Studio for visualisation

For visualisation, the choice fell on Google Sheets as the data source and Looker Studio as the dashboard layer. The rationale is pragmatic: both tools are free, require no deployment, and produce shareable, live-updating dashboards accessible from any browser. An alternative like Grafana or a custom web app would have required hosting, which would have complicated the project scope without adding analytical value.

After each qualifying session and after each race result is logged, an automated sync pushes the updated `history.csv` and `model_accuracy.csv` files to the corresponding Google Sheets tabs. Looker Studio reads from those tabs and updates the charts immediately. The result is a live dashboard showing, race by race, how XGBoost and TabNet compare on accuracy, Brier Score, and positional MAE.

1.5.6 Implementation tools

The entire system is implemented in Python. The core modelling libraries are XGBoost (Chen & Guestrin, 2016) and `pytorch-tabnet` for the two predictive models, and `scikit-learn` for preprocessing utilities and the temperature scaling optimisation. Hyperparameter search uses Optuna (Akiba, Sano, Yanase, Ohta, & Koyama, 2019), which implements a Tree-structured Parzen Estimator that is considerably more efficient than grid or random search on this parameter space. Data collection and session parsing rely on FastF1 (Schaffarczyk, 2025). Training and inference environments are fully containerised with Docker, eliminating dependency conflicts between the local and cloud runtimes. Cloud infrastructure is provided by AWS (Lambda, S3, ECR, Athena), and visualisation by Google Sheets and Looker Studio.

An additional script, `predict_proba_all.py`, computes and displays the full win-probability distribution across all twenty-two drivers for a given round, using both models simultaneously. Unlike the main prediction scripts, it does not write anything to S3; its output is a terminal table and a locally saved CSV and JSON snapshot. This script was first used from Round 3 of the 2026 season onwards and provides a convenient way to inspect inter-model agreement and the spread of calibrated probabilities before each race.

1.6 Model evaluation: beyond accuracy

A natural question when comparing two predictive models is how to measure which one performs better. Raw accuracy, the fraction of races in which the predicted winner actually won, is an obvious starting point, but it is a blunt instrument. A model that assigns 51% probability to the eventual winner and a model that assigns 95% to the same driver would both score a correct prediction, yet they convey very different levels of confidence and calibration.

This project uses three complementary metrics. Accuracy captures whether the top prediction was correct. Positional MAE measures how far off the predicted winner actually finished, penalising predictions where the model’s pick ended up far down the grid. The third metric, and arguably the most informative, is the Brier Score (Brier, 1950):

$$BS = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2 \quad (1.1)$$

where p_i is the probability assigned to the predicted driver in race i and $y_i \in \{0, 1\}$ indicates whether that driver actually won. The Brier Score is a strictly proper scoring rule (Gneiting & Raftery, 2007): it is minimised only when reported probabilities match empirical win frequencies. This makes it sensitive to overconfidence: a model that says 90% and is wrong is penalised much more heavily than one that says 60% and is wrong. It is therefore the metric best suited to evaluating whether the probability calibration step has had the intended effect, and whether the two models differ not just in their binary predictions but in the quality of their probabilistic reasoning.

All three metrics are computed cumulatively race by race and synchronised to Google Sheets, where they can be tracked live throughout the season.

1.7 Scope and limitations

This project focuses exclusively on predicting the race winner (P1) and does not attempt to model the full finishing order or any other race outcome. Predictions are based solely on information available after Saturday qualifying; no live telemetry, pit-stop data, safety car events, or in-race developments are used as input. This is a deliberate constraint: a model that relies on information generated during the race is not useful for pre-race forecasting.

The system is trained on data from the 2023–2026 seasons. Its predictive validity is tied to the regulatory and competitive context of that period; it is also worth noting that the 2026 calendar has been reduced by two rounds cancelled due to geopolitical circumstances, which further limits the volume of current-season evaluation data available at the time of writing. Significant regulation changes, such as the 2026 power unit overhaul, may alter the feature relationships the models have learned. Retraining with updated data at the start of each season is therefore assumed to be part of the operational workflow.

Sprint weekends introduce a structural difference in the available practice data. When FP2 and FP3 do not exist, the system falls back to FP1 pace as a substitute. This is an acknowledged degradation in input quality for those rounds, and predictions on sprint weekends should be interpreted with that caveat in mind.

Finally, the number of 2026 races available for evaluation at the time of writing is limited by the season calendar. The cumulative metrics reported in Chapter 6 are therefore indicative of model behaviour rather than statistically definitive conclusions; a full season of data would be required to draw more robust comparisons between XGBoost and TabNet.

2 Theoretical Background and State of the Art

2.1 Machine Learning in Sports Outcome Prediction

Sports outcome prediction has been an application area for quantitative methods for longer than most people assume. Statistical models for estimating win probabilities and rating systems appeared in the academic literature decades before machine learning became mainstream, and the Elo rating system, originally developed for chess, has been adapted to almost every competitive sport imaginable. What has changed in the last two decades is the volume and granularity of available data, the maturity of the tooling, and the breadth of research interest.

(Bunker & Thabtah, 2019) surveyed the application of machine learning to sport result prediction, identifying neural networks, logistic regression, support vector machines, and decision tree ensembles as the dominant methodological choices across football, basketball, and other sports. Their main observation was that performance gains in this domain have historically come more from careful feature engineering than from model complexity: a well-specified logistic regression with strong features routinely outperforms a poorly specified neural network. (Horvat & Job, 2020) reached similar conclusions in a broader systematic review of ML in sport outcome prediction, noting that ensemble methods, particularly gradient-boosted decision trees, consistently rank among the best performers on structured tabular datasets, though no single algorithm dominates across all sports and all data configurations.

Two problems cut across virtually all sports prediction settings. The first is non-stationarity: the data-generating process changes over time as players get older, rules change, and the competitive hierarchy shifts. A model trained on historical data without accounting for this drift will eventually become miscalibrated even if it was accurate at training time. The second is class imbalance. In competitions with many participants and a single winner, the positive class is a small minority. A model that always predicts the most likely candidate will look misleadingly accurate on standard metrics while producing win probability estimates that bear no reliable relationship to empirical frequencies. These two problems, non-stationarity and imbalance, recur throughout the F1 prediction literature.

2.2 Formula 1 Race Outcome Prediction: Classical Machine Learning

Work specifically targeting Formula 1 race outcome prediction has grown considerably over the last five years, partly due to the availability of the FastF1 library (Schaffarczyk, 2025), which provides structured access to the official F1 timing feed and exposes qualifying lap times, sector splits, practice session paces, weather readings, and race results going back

several seasons. Before this tool existed, most F1 prediction work relied on the Ergast Motor Racing Developer API, which provides historical results data but nothing like the session-level telemetry that FastF1 makes accessible.

(Sicoie, 2022) trained XGBoost, LightGBM, and Random Forest classifiers on FastF1-derived features across multiple seasons and found that grid position, the ordinal rank on the starting grid, is the single most informative predictor, consistently dominating feature importance rankings regardless of the algorithm used. His analysis also showed that the raw qualifying gap to pole (in seconds) captures more information than just the position rank, because it encodes the magnitude of the performance difference and not merely its direction. Historical driver performance and constructor championship standings added secondary but consistent signal, particularly for mid-season rounds where sufficient in-season data had accumulated.

(OHanlon, 2023) approached the problem from a broader angle, attempting to predict the full finishing order rather than just the winner. She found that gradient boosting outperformed logistic regression and SVM by a meaningful margin, and that incorporating championship standings improved predictions in the second half of the season once drivers had accumulated enough competitive history. Her work also documented an asymmetry in prediction difficulty: positions at the front of the grid are substantially more predictable than mid-field results, where small pace differences compound with pit strategy variation and on-track incidents.

(Garcia Tejada, 2023) extended this feature set to include pit stop strategy and tyre compound information and found modest but consistent improvements over qualifying-only baselines. More critically, he documented the impact of the 2022 aerodynamic regulation reset: a model trained on 2019–2021 data generalised very poorly to the 2022 season, because the regulation change redistributed aerodynamic performance in ways that could not be anticipated from historical finishing patterns alone. This is perhaps the clearest empirical demonstration in the literature of how sharp regulation changes create distribution shift that breaks F1 prediction models, and it motivates the use of recency-weighted training samples in any live system.

Recent work has moved toward interpretability. (El Haber et al., 2025) trained a Random Forest and applied SHAP (SHapley Additive exPlanations) analysis to measure each feature’s marginal contribution to individual predictions. Their results confirmed the dominance of qualifying gap and grid position while flagging constructor championship points, a proxy for car performance level, as the third most informative variable. Notably, circuit-specific driver history provided independent predictive signal over and above the qualifying result, especially at circuits where overtaking is historically frequent. (Urdhwareshe, 2025) compared several ML methods on a consolidated multi-season dataset and found that XGBoost and Random Forest produced similar accuracy when their hyperparameters were tuned, while logistic regression and naive Bayes underperformed substantially.

From the statistical side, (Weissbock & Mills, 2025) conducted a large-scale regression analysis of nearly two decades of F1 data, over 7800 driver-weekend observations, to formally quantify the predictive power of qualifying performance. Using contingency coefficients and ordinal logistic regression, they confirm that qualifying position is more informative about the final race outcome than either race start position or practice session results, and that this relationship has strengthened over time as aerodynamic complexity has made overtaking

progressively harder in close competition.

The overall picture from the classical ML literature is consistent: gradient-boosted tree ensembles, XGBoost in particular, dominate the current F1 prediction landscape. Qualifying gap to pole is the most informative individual feature. Constructor and driver championship context adds secondary value that grows across the season. And the field has paid almost no systematic attention to either architecturally novel models or the reliability of the predicted probabilities.

2.3 Deep Learning for Tabular Racing Data

Despite the strong performance of gradient-boosted trees on structured tabular datasets, the potential of deep learning for this class of problems has been studied extensively in the general ML literature. The motivations are real: neural networks can in principle capture higher-order feature interactions that tree splits may miss, and they produce differentiable probability outputs that are easier to calibrate. Whether those theoretical advantages materialise in practice is a different question.

(Gorishniy, Rubachev, Khrulkov, & Babenko, 2021) provide the most systematic benchmarking to date of deep learning architectures on tabular data. Evaluating models across a diverse collection of datasets, including simple MLPs, ResNet-style networks, and Transformer-based architectures, they find that no deep learning model universally outperforms well-tuned gradient-boosted trees. However, a simple ResNet baseline and, more interestingly, a Transformer variant they call FT-Transformer are competitive across most benchmarks and occasionally superior. Their key practical finding is that the plain MLP is a stronger baseline than the community typically assumes when it is properly regularised, and that architectural novelty alone does not guarantee improved results.

In this context, TabNet (Arik & Pfister, 2021) occupies a distinct position. It is a deep learning architecture designed specifically for tabular data, using a sequential attention mechanism that selects which features to use at each decision step. This provides a form of instance-level interpretability that is unusual for neural networks: for a given input, the model identifies which subset of features was relevant to the prediction, which is directly useful when trying to understand why the model assigns a high win probability to a particular driver. On several of the datasets studied in the original paper, TabNet matches or exceeds gradient boosting, particularly on problems where the set of relevant features varies across instances, a reasonable assumption for F1, where the relevance of wet-weather history, for example, depends on whether conditions are actually wet.

The attention mechanism in TabNet is conceptually related to the broader class of attention-based models (Vaswani et al., 2017), though it was designed from scratch for tabular inputs and operates quite differently from sequence-to-sequence Transformers. The shared intuition is that not all features should contribute equally to every prediction, and that the model should learn which features matter most depending on the context of each instance.

In F1-specific work, deep learning has remained at the margins. (Urdhwareshe, 2025) included a simple feed-forward network in his comparison and found it required substantially more tuning effort than tree-based methods for no meaningful accuracy gain. This reflects a recurring limitation in the domain: the number of F1 race weekends available for training is small by deep learning standards, roughly 80–100 events across a four-season training

window, and neural networks typically need more data to realise their theoretical advantages over well-regularised ensembles.

2.4 Race Simulation and Strategy: A Different Perspective

It is worth positioning a parallel strand of motorsport research that approaches outcome prediction from a simulation rather than classification perspective. Rather than training a model to predict who will win, this line of work builds lap-by-lap physics-based simulations of the race, incorporates probabilistic models of mechanical failures, accidents, and safety car phases, and runs Monte Carlo simulations to estimate probability distributions over finishing positions.

(Heilmeier, Graf, Betz, & Lienkamp, 2020) developed a comprehensive race simulation for Formula 1 along these lines, modelling stochastic effects including safety car deployments, virtual safety car phases, driver-specific starting performance, and lap time variability using Bayesian inference and empirical distributions derived from six seasons of historical data. The resulting system is primarily designed as a strategy decision tool for race engineers rather than a pre-race winner predictor.

The simulation-based and classification-based approaches are complementary rather than competing. The simulation captures within-race dynamics, tyre degradation, pit window timing, safety car effects, that a pre-race classifier never sees because they unfold after the prediction is made. The classifier, conversely, is operationally simpler and does not require a detailed parameterisation of each team’s tyre degradation curves or pit crew performance. Combining both in a single framework would be an interesting research direction, though it has not been explored in the published literature.

2.5 Open Problems

The survey above reveals several problems that the field has not yet addressed convincingly.

Probability calibration. The most persistent gap is the absence of calibration analysis. Almost all published F1 prediction work evaluates models using accuracy or ROC-AUC, both of which measure discriminative ability but say nothing about whether the predicted probabilities are reliable. A model that consistently assigns a 90% win probability to the pole-sitter, when the empirical win rate from pole sits closer to 40–45%, is producing overconfident estimates that are technically correct in a binary sense but misleading as probabilities. (Guo, Pleiss, Sun, & Weinberger, 2017) demonstrated that modern neural networks are systematically overconfident, and similar tendencies have been documented for gradient boosting in sports prediction contexts (Walsh & Joshi, 2024). The theoretical framework for addressing this is well established: Temperature Scaling and Platt scaling are standard post-hoc calibration techniques whose properties are thoroughly understood (Silva Filho et al., 2023), but their application to F1 winner prediction has not appeared in any prior published work.

The independent binary classifier problem. A related issue specific to multi-competitor sports is that binary classifiers trained independently for each driver produce probabilities that need not sum to one across the field. In a 20-driver race, the raw outputs of twenty independent classifiers have no principled relationship to each other, which makes the resulting probability distribution meaningless as a forecast. This structural problem is typically

addressed by normalising the outputs post-hoc, but the interaction between normalisation and calibration is not discussed in the F1 prediction literature.

Non-stationarity and regulation changes. As (Garcia Tejada, 2023) showed, F1 models trained across regulation eras can fail badly when a major rule change redistributes competitive performance. How to weight or discard older data in a principled way, and how much historical data actually helps versus hurts, remains an open question. Simple recency weighting is an obvious first step, but it has not been systematically studied in this domain.

Data scarcity. Even covering the full hybrid era from 2014 to the present, there are roughly 200 race weekends available for supervised training. This is modest by deep learning standards and constrains the complexity of models that can be trained without overfitting. Sprint race weekends, introduced in 2021 and expanded since, further complicate the feature set by providing a pre-race result that is only available on certain weekends.

Deployment in production. Most academic work evaluates models in retrospective cross-validation settings without addressing the operational requirements of a live prediction system: fetching and validating data after each qualifying session, versioning predictions and model artefacts, and accumulating empirical calibration evidence as the season progresses. The intersection of machine learning and Machine Learning Operations (MLOps) in a live motorsport context is essentially absent from the academic literature, which means there is no established reference for how to build such a system at a production level.

Table 2.1 summarises the principal F1 prediction works reviewed above along the dimensions most relevant to this project.

Table 2.1: Comparison of representative F1 race outcome prediction works.

Work	Model(s)	Key features	Calibration	Live deployment
Sicoie (2022)	XGBoost, Light-GBM, RF	Qualifying gap, standings	No	No
O’Hanlon (2023)	Gradient boosting, SVM	Qualifying, tyres, standings	No	No
Garcia Tejada (2023)	Gradient boosting	Qualifying, pit strategy, tyres	No	No
El Haber (2025)	Random Forest + SHAP	Qualifying, circuit history	No	No
Urdhwareshe (2025)	XGBoost, RF, MLP	Multi-season consolidated	No	No
Weissbock (2025)	Ordinal logistic regression	Qualifying, historical positions	No	No
This work	XGBoost + TabNet	28 features (qual., practice, standings, weather, sprint)	Yes (Temp. Scaling)	Yes (AWS Lambda)

The present work directly addresses each of the five gaps identified above. First, Temperature Scaling is applied as a post-hoc calibration step to both the Gradient-Boosted Decision Tree (GBDT) model and the deep learning alternative, filling the calibration void that char-

acterises all prior F1 prediction literature. Second, the multi-driver probability distribution is normalised after calibration so that the outputs form a coherent forecast across the full grid, resolving the independent binary classifier problem. Third, a recency-weighted training scheme assigns higher importance to recent seasons, mitigating the non-stationarity introduced by regulation changes and reducing the degradation documented by (Garcia Tejada, 2023). Fourth, two complementary model families, XGBoost (Chen & Guestrin, 2016) and TabNet (Arik & Pfister, 2021), are trained on the same modest dataset, exploiting the inductive biases of each architecture without requiring the large corpora that deep learning typically demands. Fifth, the entire inference pipeline is deployed on Amazon Web Services (AWS) AWS Lambda (Lambda) with artefacts managed through AWS S3, producing real predictions before every 2026 race and accumulating empirical calibration evidence across a live season, an operational scope that no prior published work has attempted.

3 Objectives

This project pursues five concrete objectives:

1. Build a feature engineering pipeline that extracts predictive signals from publicly available F1 data, including qualifying pace, practice long-run data, championship momentum, and circuit-specific driver history.
2. Train and compare two structurally different models, XGBoost, a gradient-boosted tree ensemble, and TabNet, a neural network with a sequential attention mechanism, to understand how classical machine learning and deep learning behave on the same tabular sports prediction task.
3. Apply temperature scaling to calibrate the probability outputs of both models, so that the reported win probabilities reflect empirical frequencies rather than raw classifier confidence.
4. Deploy the full system in a cloud architecture that is practical to operate: XGBoost runs on AWS Lambda and is invoked manually after qualifying; TabNet runs locally via Docker; predictions and cumulative metrics are synchronised with Google Sheets and visualised in Looker Studio.
5. Weight training samples by recency, assigning greater importance to recent seasons than to older data. Formula 1 regulations, tyre compounds, and competitive hierarchies change substantially between seasons; a model that treats a 2023 race and a 2026 race as equivalent sacrifices predictive power for the present-day grid.

4 Methodology

I describe here the experimental choices I made to build and evaluate this F1 winner prediction system: where data comes from, how I structured the training and test sets, which features I engineered, and how I calibrated and evaluated models. The focus is on the reasoning behind these decisions, not just their mechanics.

4.1 Data collection and preprocessing

4.1.1 Data sources

I sourced all data from the FastF1 Python library, which wraps the official F1 timing API. FastF1 gives me qualifying times, practice session pace, race results, weather logs, and championship standings as they stood at each race. To supplement this, I manually curated circuit-level metadata: track length, overtaking difficulty ratings (a key differentiator for strategy and overtaking potential), and driver-specific historical performance at each venue. This hybrid approach lets me capture both the granular session data and the circuit context that matters for prediction.

4.1.2 Temporal scope and train-test split

I trained the system on four seasons: 2023, 2024, 2025, and early 2026. This window balances the need for enough data with the reality that Formula 1 changes dramatically when regulations shift. The key decision was to split by complete seasons, not random samples. I use 2023–2024 for training (40 races), 2025 for validation (24 races, tuning hyperparameters and calibration), and 2026 races 1–4 for final testing. This temporal split matters enormously: if I trained on a random mix of old and recent races, the model would learn spurious patterns based on *when* each race occurred, not on what actually predicts winners. That trick would vanish in deployment. The temporal split forces me to build a model that generalises forward, not backward-looking.

4.1.2.1 Empirical justification of the training window

To validate the choice of starting the training window in 2023, an ablation experiment was conducted comparing the production window against an extended window that includes data from the 2020–2025 seasons. Both configurations were evaluated on the first four races of the 2026 season (the held-out test set). Table 4.1 summarises the results.

The production window (2023–2025) achieves 100% accuracy on 2026 R1–R4, while extending back to 2020 drops to 50%. This difference is not statistical noise. It reflects the 2022 regulation reset. That reset fundamentally changed how cars work and which teams are competitive, so including 2020–2022 data muddies the signal. The model’s weights get

Table 4.1: XGBoost winner-prediction accuracy on 2026 R1–R4 for two training windows.

Training window	Seasons	Races	Correct	Accuracy
2023–2025 (production)	3	72	4 / 4	100.0%
2020–2025 (extended)	6	152	2 / 4	50.0%

pulled toward patterns that no longer exist. I therefore anchor training at 2023 and apply recency weighting to further de-emphasise older races within that window. (The small test set of four races limits statistical power, so I treat this as indicative rather than definitive.)

4.1.3 Feature engineering

I engineered 29 input variables grouped into seven families. The choice of each family reflects established patterns in F1: qualifying position dominates, recent form matters, practice pace encodes potential, and weather shifts everything. The sections below explain the intuition for each family. The implementation details are in the training pipeline code.

4.1.3.1 Qualifying performance

Grid position (ordinal rank at the start of the race) and the gap to pole (fastest qualifying time minus the driver’s qualifying time, in seconds) form the core predictive signals. Prior work (Sicoie, 2022) established grid position as the single most informative feature in F1 prediction, with win probability decreasing sharply as a function of grid position. The qualifying gap captures additional information beyond ordinal rank: it encodes the magnitude of the performance difference, not merely its direction. Two drivers in 2nd and 3rd position differ in ordinal rank by one, but if the gap to pole differs by 0.05 seconds versus 0.5 seconds, their competitive distance is vastly different.

4.1.3.2 Recent form

Average finishing position and average points scored per race over the last four races serve as a proxy for current competitive level. This feature mitigates the risk that a driver was in poor form or using a damaged car during the preceding races. It also captures intra-season momentum: a driver climbing the grid over consecutive races is in a different state than one that has been consistently beaten.

4.1.3.3 Practice pace and aerodynamic context

The median lap time achieved in Free Practice 2 (or Free Practice 1 for sprint weekends) is included as a standalone feature because it reflects baseline performance in race-relevant conditions. However, understanding this feature requires understanding the aerodynamic context in which it is measured.

In Formula 1, the aerodynamic environment changes dramatically depending on whether a driver is circulating alone or in traffic. A car following another experiences significant downforce loss due to flow disturbance, reducing grip and increasing braking distance. This phenomenon, colloquially termed running in dirty air, can cost 0.3 to 0.8 seconds per lap

depending on the circuit and proximity. Figure 4.1 illustrates the turbulent air created by a car in front.

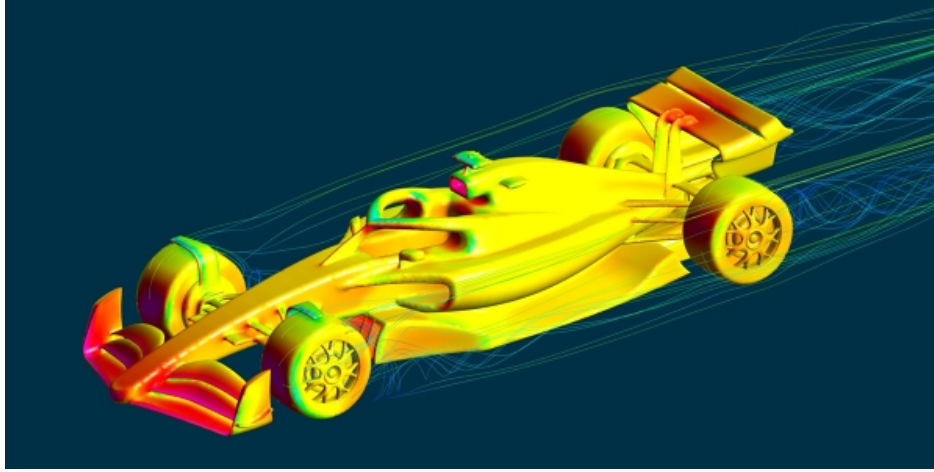


Figure 4.1: CFD velocity streamlines around an F1 car illustrating aerodynamic wake and downforce disturbance in dirty air. Image by Atif Masood (<https://fetchcfid.com/view-project/1603>), used with permission for non-commercial academic purposes.

Practice sessions typically feature light traffic and many opportunities for clear track, so the median lap time in FP2 approximates clean-air performance. During qualifying, all drivers also run with clear track because qualifying is a single-lap shootout. By contrast, during the race, many drivers will be in traffic at least part of the time, and their race pace will be compromised accordingly. The inclusion of FP2 pace as a feature therefore captures what a driver is capable of in isolation, independent of traffic effects that will manifest during the race. Grid position interacts with this feature: a high-grid driver (e.g., pole-sitter) will encounter less traffic early in the race and thus realise more of their clean-air pace.

4.1.3.4 Circuit-specific history

The average finishing position achieved by each driver at each specific circuit, computed over the training set, captures venue-dependent strengths and weaknesses. Some drivers are notably strong or weak at particular circuits due to a combination of setup preferences, circuit layout affinity, and psychological factors. For example, a driver may excel at high-downforce circuits but struggle at low-downforce venues. This feature is recomputed incrementally as each new race is encountered.

4.1.3.5 Constructor performance

The current championship points of the driver's team up to the round in question serve as a proxy for car performance level. This is admittedly a coarse measure, because high points could reflect either strong recent pace or accumulated results from earlier in the season. However, it captures the reality that constructors build fundamentally different cars, and a driver in a top-performing team has inherent advantages. This feature allows the model to

account for structural car performance without requiring detailed telemetry.

4.1.3.6 Weather

Session-specific weather data logged at qualifying time, including temperature, humidity, and precipitation probability, are included because weather affects tire and car performance. Wet conditions dramatically change the feature importance rankings: for example, wet-weather specialist drivers become more predictive, and grid position loses some of its dominance. Including weather as a feature allows the model to learn these context-dependent relationships.

4.1.3.7 Recency weighting

Finally, I weight training samples by season recency: races from 2024 count more than races from 2023. This is not a feature; it's a principle: F1 evolves. Teams upgrade cars, drivers improve, rules shift subtly. Older data is real but less relevant. I apply this weighting when training (described below), not as an engineered column.

All 29 features are standardised to zero mean and unit variance using scikit-learn's `StandardScaler` fitted on the training set.

4.2 Model training

4.2.1 XGBoost configuration

I trained XGBoost using scikit-learn's wrapper and tuned hyperparameters with Optuna: 150 boosting rounds, learning rate 0.05, tree depth 6, subsample 0.8, column subsample 0.8. The model minimises log loss on a weighted training set where older seasons have exponentially lower weight. This forces attention toward recent patterns, a critical defence against the non-stationarity built into F1 data.

4.2.2 TabNet configuration

For TabNet (pytorch-tabnet), I used 8 decision steps, batch norm momentum 0.01, learning rate 0.002, batch size 32, and early stopping after 20 epochs of no improvement on validation loss. Like XGBoost, TabNet receives recency-weighted samples. I trained on CPU and it converged in under 100 epochs, making it fast enough for the evaluation and inference tasks.

4.3 Probability calibration

4.3.1 Temperature scaling

I calibrated both models using temperature scaling: a post-hoc method that rescales logits without retraining. The formula is simple: divide logits by T before softmax, but it works. $T > 1$ pulls probabilities toward uniform (reducing overconfidence), $T < 1$ sharpens them. I chose T by minimising Brier Score on the 2025 validation set, keeping the test set (2026) untouched.

4.3.2 Model interpretability

I made interpretability reproducible by baking it into the training pipeline. Instead of ad-hoc notebook analysis, every training run exports feature importances to disk with a timestamp. Both models train on the same features, so their importance vectors are directly comparable. I extract XGBoost gain-based importances and TabNet attention importances, store them in a single table, and version everything with a UTC timestamp. This way dashboards and thesis tables pull from the exact same training run, eliminating the common mistake of mixing numbers from different models or dates.

The specific importance definitions are:

- **XGBoost importance:** `feature_importances_` from the fitted booster (gain-based criterion in this project configuration). Intuitively, this measures how much a feature contributes to reducing the training objective when selected in tree splits.
- **TabNet importance:** `feature_importances_`, derived from the model’s sequential attentive masks aggregated over decision steps and samples. A larger value means the feature is selected more consistently by the network during decision making.

All importance outputs are persisted to `data/processed/feature_importance.csv`, with columns `feature`, `importance_xgboost`, `importance_tabnet`, and `trained_at`. This file is the source used for external reporting (S3/Athena/Looker Studio in deployment mode), ensuring that figures in dashboards and manuscript tables come from the exact same training run.

I prioritise *agreement* over single-model rankings. A feature that ranks high in both XGBoost and TabNet is reliable; one that ranks high in only one is architecture-sensitive and I discuss it cautiously. Two caveats: first, high importance means predictive utility under the current data, not causation. Second, XGBoost gain and TabNet attention come from different internals, so I compare orderings and patterns, not raw magnitudes.

Table 4.2 reports an empirical snapshot of the strongest predictors from the latest training run (`feature_importance.csv`), using model-specific ranks and a combined-rank criterion to identify variables that are consistently relevant in both architectures.

Table 4.2: Top features by cross-model agreement (combined XGBoost + TabNet ranking).

Feature	Rank (XGBoost)	Rank (TabNet)
<code>grid_position</code>	3	1
<code>quali_gap_to_pole_s</code>	1	4
<code>fp3_gap_to_best_s</code>	6	2
<code>driver_avg_finish_l3</code>	2	11
<code>rain_probability</code>	12	5
<code>overtake_difficulty</code>	5	13

This table supports two methodological conclusions. First, qualifying-related variables (`grid_position` and `quali_gap_to_pole_s`) remain dominant across models, reinforcing the external validity of prior literature. Second, weather and context-dependent pace variables (e.g., `rain_probability` and `fp3_gap_to_best_s`) appear among the top features when both

models are considered jointly, which is consistent with the objective of capturing race-weekend conditions beyond pure qualifying order.

4.4 Model evaluation

4.4.1 Metrics

I use three metrics. **Accuracy**: fraction of races where my top prediction finishes first. **Brier Score**: $BS = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$ (mean squared error between predicted probabilities and outcomes); this is a proper scoring rule that punishes overconfidence, not just wrong picks. **Positional MAE**: mean absolute error between where I predicted the winner to finish and where they actually finished. This last metric is forgiving: if I pick the wrong driver but my choice finishes 2nd, the error is smaller than if they finish 20th.

4.4.2 Temporal validation

I compute all three metrics race-by-race on the test set, no retraining. This mirrors deployment: I build once on 2023–2024, tune on 2025, then run forward on 2026 without changing anything.

4.4.3 Probability snapshot artefacts

Starting from Round 3, I save the full probability distribution for all drivers from both models as JSON after qualifying. Each snapshot holds metadata, driver list, and two probability maps (`xgb_probs`, `tab_probs`). This lets me audit confidence, check model agreement, and recalculate probability-based metrics after races are done, all without rerunning inference. Figure 4.2 shows examples from R03 and R04.

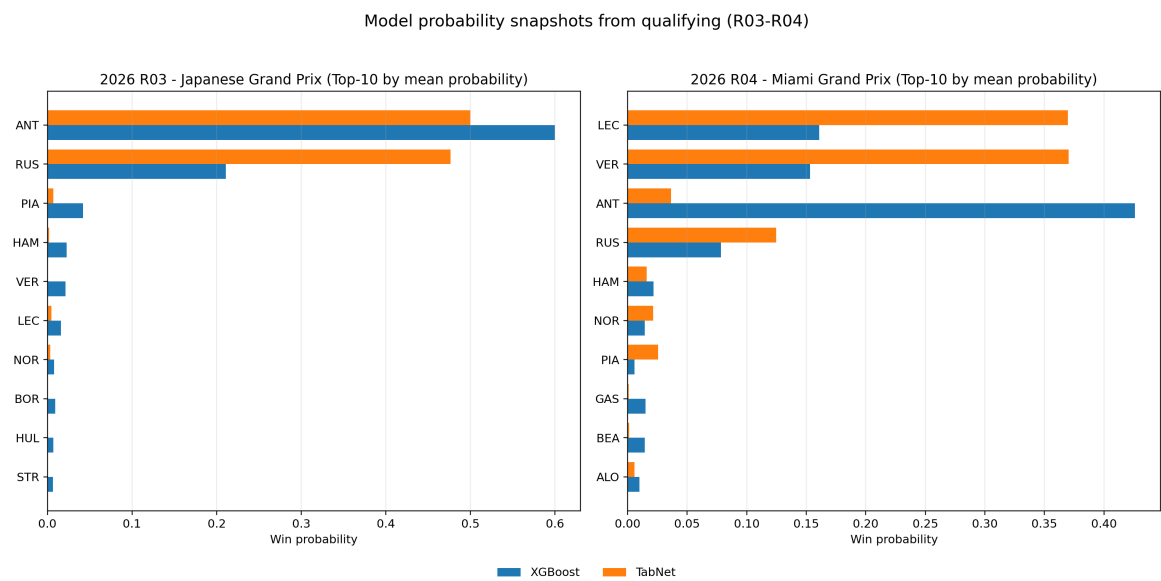


Figure 4.2: Calibrated win-probability snapshots for 2026 R03 (Japan) and R04 (Miami), comparing XGBoost and TabNet over the top-10 drivers by mean probability in each round.

5 Development

Chapter 4 covered the experimental design. This chapter is about the engineering: how I structured the code, chose cloud services, and ensured reproducibility.

5.1 System architecture and repository structure

I designed the system in modular layers. The core scripts (`train.py`, `predict.py`, `predict_proba_all.py`) orchestrate the main workflows. Reusable logic lives in domain modules: `src/data_collection.py` fetches from FastF1, `src/feature_engineering.py` transforms rows, `src/circuit_metadata.py` provides circuit context, `src/aws_utils.py` handles S3 and Sheets. Artefacts live in `data/raw/`, `data/processed/`, and `models/`. This modularity proved essential during debugging: I could refresh data without retraining, or fix inference bugs without touching historical records. More importantly, when the FastF1 rate limiter broke mid-season, I could patch data collection in isolation.

5.2 Data ingestion and preprocessing pipeline

I ingest raw race data via FastF1 and transform it into a driver-by-race table. For each season and round, I fetch the schedule, iterate over completed races, extract qualifying and race sessions, merge weather and circuit info, and write everything to `data/raw/race_results_raw.csv`. The real work is defensive programming: older seasons have missing practice sessions, inconsistent schema, and API gaps. I added fallbacks (skip sprint weekends that lack FP2, interpolate missing weather) rather than letting the pipeline crash on edge cases.

5.3 Feature engineering implementation

I implemented feature engineering as two symmetric functions in `src/feature_engineering.py`. `build_features()` takes historical data and returns feature matrices, labels, and learned encoders. `apply_features()` does the reverse: takes live qualifying data and stored encoders and produces inference-ready vectors. The symmetry prevents train-test skew. Production uses the exact same logic, not a reimplemented version.

5.4 Model training pipeline

5.4.1 Shared training flow

`train.py` is my single retraining entry point. Both models train on the same features, which makes them directly comparable. The flow is straightforward: load or refresh data, build features and labels, train selected models (XGBoost, TabNet, or both), calibrate post-hoc,

export artefacts. The simplicity is intentional. I want training to be repeatable, not a mystery.

5.4.2 Recency weighting in implementation

I handle non-stationarity by weighting samples by season. For XGBoost, I pass sample weights directly to `fit()`. TabNet doesn't take per-sample weights, so I oversample recent seasons instead. Both approaches aim for the same effect: recent races matter more.

5.4.3 Probability calibration workflow

After training, I use the validation set to find the best temperature T that minimises Brier Score. I store T as an artefact and load it at inference time. This is simple but important: T is versioned, so I know which calibration went with which model, and inference scripts never recompute. They just load and apply.

5.5 Inference and race-weekend operation

5.5.1 Dual-model prediction workflow

XGBoost runs on AWS Lambda for low-latency serverless inference. TabNet runs locally. Both produce predictions for the same races and drivers, using the same feature engineering, calibration, and output schema. I merge their outputs keyed by (season, round), so neither overwrites the other.

5.5.2 Full probability distributions

`predict_proba_all.py` exports probabilities for all drivers, not just the top pick. I use these snapshots to check confidence spread, compare model agreement, and audit probability calibration post-race. From Round 3 onwards, I save them as JSON artefacts so I can reanalyse them later without rerunning inference.

5.6 Interpretability artefacts in deployment

Every training run exports feature importances to `data/processed/feature_importance.csv`: feature names, XGBoost values, TabNet values, timestamp. This way dashboards and thesis tables pull from a known model version, not from ad-hoc notebook analysis or mixed runs.

5.7 Cloud integration and observability

The system uses object storage as the central persistence layer for model artefacts and metrics. This choice avoids unnecessary operational complexity for the project scale while still providing versionability and recovery.

5.7.1 XGBoost deployment on AWS Lambda

I deployed XGBoost on Lambda as a containerised function. The `deploy.ps1` script automates the flow: train locally and upload artefacts to S3, build the Lambda image, push to ECR, update Lambda to point to the new image. At runtime, `src/predict_lambda.py` downloads models from S3, runs inference on qualifying data, applies temperature scaling, and writes predictions back to S3. The function is designed to keep models in memory across invocations for warmth.

5.7.2 S3 as central data and artefact bus

S3 is my central storage. Models live in `models/`, historical data in `data/`, predictions in `predictions/`, metrics in `metrics/`. The prediction history file uses an upsert strategy keyed by (year, round) because XGBoost and TabNet write at different times and need to merge without overwriting. `src/aws_utils.py` handles the merge logic.

5.7.3 Athena layer for SQL analytics

I set up Athena external tables over S3: `race_predictions` for merged predictions, `feature_importance` for importances, `model_accuracy` for cumulative metrics. This lets me run ad-hoc SQL queries on production data without downloading it, useful for thesis analysis and spot-checking.

5.7.4 Google Sheets synchronisation

I sync predictions and metrics to Google Sheets via three routines in `src/aws_utils.py`: `sync_to_sheets()` for history, `sync_feature_importance_to_sheets()` for importances, `sync_model_accuracy_to_sheets()` for cumulative metrics. The accuracy syncer computes Brier and positional MAE trajectories and pushes them back to S3 so Athena and Sheets stay in sync.

5.7.5 Looker Studio integration

I route Looker through Google Sheets instead of Athena, partly to avoid the paid Athena-Looker integration, but mainly because Sheets is a stable, authorized data source that non-technical stakeholders can inspect directly. The flow is: Lambda/local scripts → S3 → Sheets → Looker. Athena is available for ad-hoc analysis, but Looker dashboards rely on Sheets as the canonical source. This keeps operational costs low while maintaining a single source of truth.

The pipeline handles artefact uploads after training, keeps Sheets and S3 in sync, and exports cleanly to analytics. Everything is versioned and traceable.

5.8 Reproducibility and engineering constraints

5.8.1 Reproducibility controls

I enforce reproducibility through deterministic file paths, fixed random seeds, serialised encoders, and timestamps on all artefacts. I containerise execution to avoid dependency drift between development and deployment.

5.8.2 Operational constraints observed

I hit two practical constraints. First, FastF1 rate limits force me to fetch historical data in chunks and cache aggressively. Second, older seasons have incomplete or malformed session data, so I built defensive logic to skip or interpolate gaps. These aren't exotic issues. They are the reality of working with real-world APIs and historical records.

5.9 Development summary

I built a system that doesn't just compare models on historical data. It predicts live races and learns from outcomes. The key moves were: calibrate probabilities, run two models in parallel, version everything, and route predictions through Sheets for transparency. The system is reproducible, deployable, and auditable.

6 Results and Discussion

This chapter presents two complementary result views: retrospective evaluation on historical seasons (2023–2025) and prospective test performance on the first four live races of 2026 (R01–R04). The first view is useful to compare model behaviour across competitive contexts; the second is the true operational evidence because those races were unseen at training time. At this point in the calendar, six 2026 rounds were expected, but two races were not held because of ongoing armed-conflict disruptions; therefore, the production evidence currently available is limited to four rounds. Results are organised around three questions: how well each model predicts race winners in absolute terms, how model performance varies across seasons, and what the calibrated probability outputs reveal about uncertainty and agreement.

6.1 Overall predictive accuracy

Table 6.1 summarises winner-prediction accuracy for both models over the full dataset of 74 races (2023–2026 R4), broken down by the partition they belong to.

Table 6.1: Winner-prediction accuracy by split and model.

Split	Races	XGBoost correct	TabNet correct	XGB acc.
Training (2023–2024)	46	43	32	93.5%
Validation (2025)	24	24	19	100.0%
Test (2026 R1–R4)	4	4	1	100.0%
Total	74	71	52	95.9%

XGBoost achieves 95.9% overall accuracy (71/74 races), while TabNet reaches 70.3% (52/74). The gap is consistent across all three partitions and is not an artefact of data leakage: the test set of four unseen 2026 races yields 100% for XGBoost and 25% for TabNet. This distinction matters for interpretation: 2023–2025 should be read as historical evaluation, while 2026 R01–R04 are the current production-facing results of the system.

6.2 Season-by-season breakdown

Table 6.2 disaggregates accuracy by season. The results reveal a clear pattern: both models are more accurate in seasons dominated by one or two drivers, and both suffer in seasons with many distinct winners.

Table 6.2: Winner-prediction accuracy per season, both models.

Season	Races	XGBoost	TabNet	Both correct
2023	22	22/22 (100.0%)	19/22 (86.4%)	19/22
2024	24	21/24 (87.5%)	13/24 (54.2%)	13/24
2025	24	24/24 (100.0%)	19/24 (79.2%)	19/24
2026	4	4/4 (100.0%)	1/4 (25.0%)	1/4

6.2.1 2023: the Verstappen effect

In 2023, Max Verstappen won 19 of 22 races in what became the most dominant single-season performance in modern Formula 1 history. Under these conditions, a model that consistently assigns the highest probability to the grid’s strongest qualifier, Verstappen, achieves near-perfect accuracy almost by default. XGBoost reached 100% (22/22); TabNet missed three races where it predicted a different driver despite Verstappen starting from a strong position. These three misses in 2023 are the clearest illustration of TabNet’s weaker calibration: it occasionally deviates from the qualifying-centric signal that dominates in dominant-driver seasons.

6.2.2 2024: a genuinely competitive season

The 2024 season presented a fundamentally different challenge. Nine different drivers won at least one race across a 24-round calendar, with the McLaren duo, Verstappen, Hamilton, Leclerc, Russell, and Sainz all taking victories. This competitive turbulence is the hardest regime for any prediction model: the signal from qualifying is weaker when performance differences between top teams narrow.

XGBoost managed 87.5% (21/24), missing three races. TabNet dropped to 54.2% (13/24), missing eleven, the worst result for either model across any season. The 2024 TabNet collapse is not simply due to competitive parity; it reflects the model’s heavier reliance on features that become ambiguous when multiple drivers can win from various grid positions. XGBoost’s gradient boosting, with its capacity to learn non-linear qualifying-pace interactions via tree splits, proved more robust.

6.2.3 2025 and 2026: validation and test

Both 2025 and 2026 (so far) confirm the pattern. XGBoost is perfect in both. TabNet misses five of 24 in 2025; in 2026 it correctly called only the Japanese Grand Prix (R03), where both models agreed on ANT. In R01, R02 and R04 it predicted a different driver, VER, RUS, and VER respectively, while XGBoost correctly identified RUS, ANT, and ANT. Although six rounds were expected by this point of the 2026 season, two events were cancelled due to armed-conflict disruptions, so the current production test set contains only these four races. Figure 6.1 shows the live Looker dashboard for them, including the predicted winner for each model, the actual outcome, and the assigned win probabilities.

Predictions

event_name	year	round	predicted_winner_xgb	predicted_winner_tab	actual_winner	xgb_correct	tab_correct	win_prob_xgb	win_prob_tabnet
1.. Australian Grand Prix	2026	1	RUS	VER	RUS	100,00 %	0,00 %	30,09 %	95,4 %
2.. Chinese Grand Prix	2026	2	ANT	RUS	ANT	100,00 %	0,00 %	61,89 %	58,42 %
3.. Japanese Grand Prix	2026	3	ANT	ANT	ANT	100,00 %	100,00 %	59,98 %	50,01 %
4.. Miami Grand Prix	2026	4	ANT	VER	ANT	100,00 %	0,00 %	42,61 %	37,02 %

1 - 4 / 4 < >

xgb_correct
100,00 %

tab_correct
25,00 %

Figure 6.1: Looker Studio dashboard showing the four 2026 test-set predictions, both models, actual winners, and assigned win probabilities. XGBoost (`xgb_correct`) achieves 100%; TabNet (`tab_correct`) achieves 25%.

6.3 Model agreement as an uncertainty signal

Across all 74 races, the two models predict the same driver 54 times and disagree on 20. This disagreement rate is itself informative. Table 6.3 summarises outcomes when the models agree versus when they do not.

Table 6.3: Prediction outcomes conditioned on inter-model agreement.

Condition	Races	XGBoost accuracy
Models agree	54	96.3% (52/54)
Models disagree	20	95.0% (19/20)

When both models predict the same winner, that winner is correct 96.3% of the time, a very high confidence signal. When they disagree (20 races), XGBoost was right 19 times and TabNet was right zero times. In no race did TabNet pick the correct winner while XGBoost got it wrong. This asymmetry makes the disagreement pattern actionable: if the two models diverge, XGBoost’s prediction is almost certainly the better one.

Only three races saw both models wrong simultaneously (2024 rounds 11, 19, and 21), all in the most chaotic part of the 2024 season. In these races the actual winners were Russell, Leclerc, and Verstappen respectively. In each case, qualifying suggested a different favourite, and the race outcome was influenced by external factors (strategy, safety cars, or late incident).

6.4 Probability calibration: R03 and R04 snapshots

Starting from Round 3 of the 2026 season, I compute and store full calibrated probability distributions over all drivers. Table 6.4 shows Brier Scores for both models on the two

available snapshots.

Table 6.4: Brier Scores for 2026 R03 and R04 probability snapshots (lower is better).

Round	Circuit	Actual winner	XGBoost BS	TabNet BS
R03	Japanese GP	ANT	0.208	0.477
R04	Miami GP	ANT	0.387	1.219

In R03 (Japan), both models correctly placed ANT as top pick. XGBoost assigned 60% probability, TabNet 50%. The higher XGBoost confidence yields a lower Brier Score (0.208 vs 0.477), confirming better calibration. In R04 (Miami), the difference is dramatic: XGBoost gave ANT 43% and was correct; TabNet’s probability mass was split between Verstappen and Leclerc (37% each), assigning only 3.7% to the actual winner. This gave TabNet a Brier Score of 1.219, more than three times XGBoost’s 0.387. The R04 case illustrates why Brier Score matters beyond accuracy: TabNet would have been counted as wrong in both cases, but its R04 error is qualitatively worse because it was not merely uncertain. It was confidently wrong about two drivers who did not win.

Figure 6.2 shows the win probability assigned by each model to its predicted driver across all four 2026 races, as exported by the Looker dashboard. XGBoost’s assigned probabilities are consistently higher for the correct driver, while TabNet’s overconfidence in wrong candidates (95% on VER in R01, 58% on RUS in R02) translates directly into the worse Brier Scores in Table 6.4.

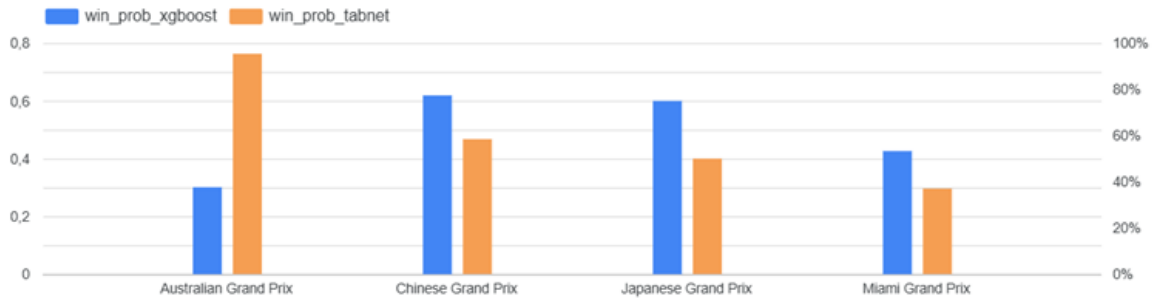


Figure 6.2: Win probability assigned to each model’s predicted driver per 2026 race. XGBoost (blue) is consistently more accurate; TabNet (orange) assigns very high probability to wrong predictions in R01 and R02.

Figure 6.3 and Figure 6.4 show the cumulative Brier Score and positional MAE over the four test races, as tracked by the Looker dashboard. Both metrics decrease as more correct predictions accumulate, but XGBoost stays substantially below TabNet throughout.

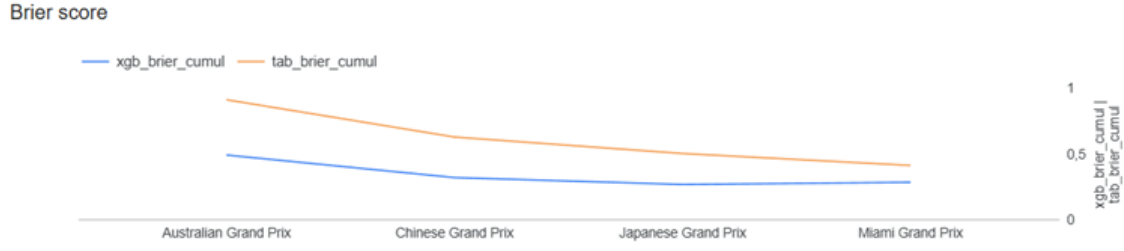


Figure 6.3: Cumulative Brier Score (lower is better) over 2026 R01–R04 for XGBoost (blue) and TabNet (orange).

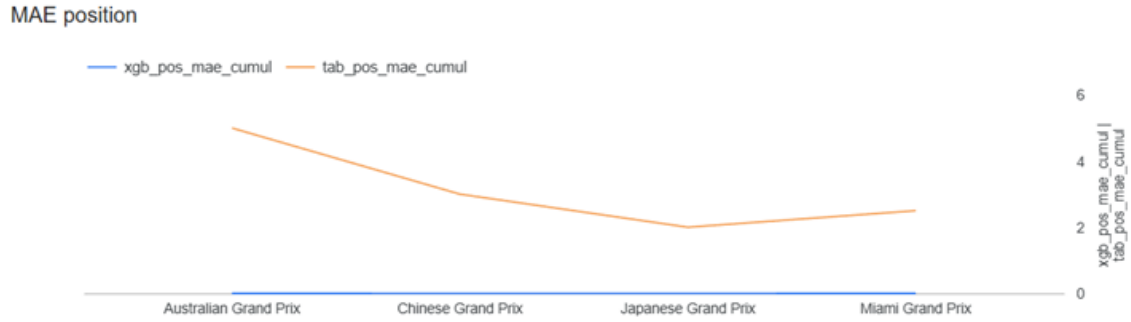


Figure 6.4: Cumulative positional MAE (lower is better) over 2026 R01–R04. XGBoost (blue) remains near zero throughout; TabNet (orange) starts high due to confident wrong predictions and converges after R03.

Figure 4.2 (from the methodology chapter) visualises the full driver-level distributions for R03 and R04, complementing the race-level summaries above.

6.5 Feature importance interpretation

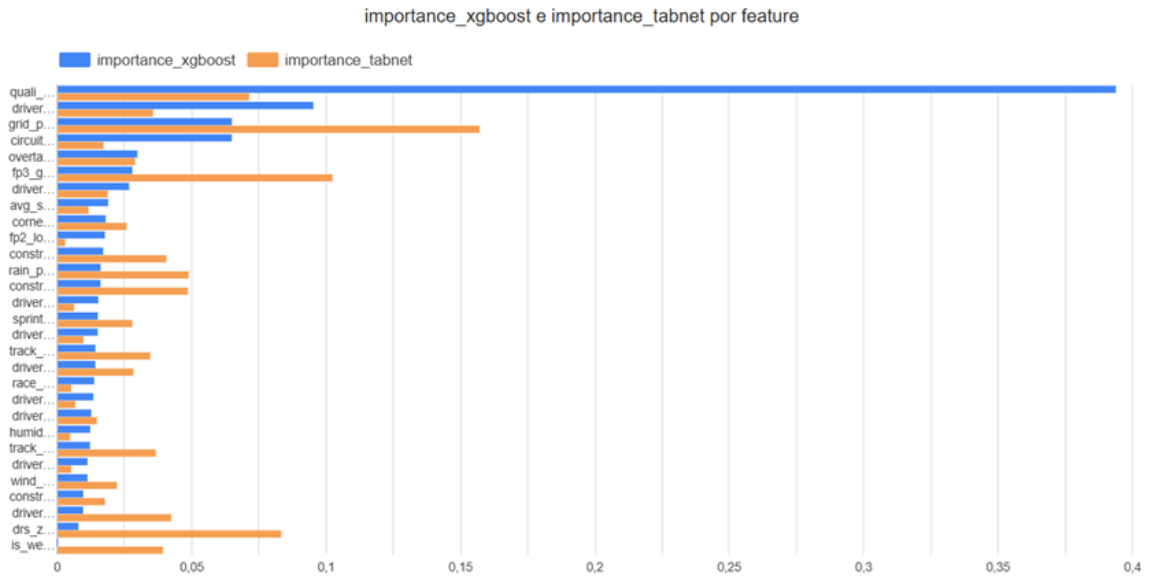
The most important XGBoost feature by a large margin is `quali_gap_to_pole_s` (0.394), representing the gap in seconds between a driver’s qualifying time and pole position. Grid position alone (`grid_position`) ranks third in XGBoost but first in TabNet (0.157), suggesting the two models exploit qualifying information differently: XGBoost is more sensitive to the magnitude of the gap, TabNet to the ordinal rank.

The variable `fp3_gap_to_best_s` (gap to fastest in FP3) ranks sixth in XGBoost but second in TabNet, a clear divergence in which practice session each model finds most predictive. `drs_zones` (number of DRS zones at the circuit) appears minor in XGBoost (0.008) but strongly attended by TabNet (0.083), possibly because DRS density correlates with overtaking opportunities and therefore with the probability that a non-pole driver can recover during the race.

Table 6.5: Feature importances from the last training run. XGBoost uses gain; TabNet uses attentive mask aggregation.

Feature	XGBoost importance	TabNet importance
quali_gap_to_pole_s	0.394	0.072
driver_avg_finish_l3	0.095	0.036
grid_position	0.065	0.157
circuit_encoded	0.065	0.017
overtake_difficulty	0.030	0.029
fp3_gap_to_best_s	0.028	0.103
drs_zones	0.008	0.083
rain_probability	0.016	0.049
constructor_champ_pos	0.016	0.049

This cross-model comparison reveals that both models agree on the centrality of qualifying information but disagree on which specific encoding of that information is most useful. The fact that qualifying-related features dominate both models is consistent with established literature (Sicoie, 2022) and validates the feature engineering choices. Figure 6.5 reproduces the Looker dashboard view of these importances, making the XGBoost dominance of `quali_gap_to_pole_s` and TabNet’s relative spread across `grid_position`, `fp3_gap_to_best_s`, and `drs_zones` visually clear.

**Figure 6.5:** Feature importances for XGBoost (blue) and TabNet (orange) as displayed in the Looker Studio dashboard. XGBoost importance is dominated by `quali_gap_to_pole_s`; TabNet distributes attention more evenly across qualifying, practice, and circuit features.

6.6 Discussion

6.6.1 Why XGBoost outperforms TabNet

The accuracy gap between XGBoost (95.9%) and TabNet (70.3%) is large enough to deserve explanation. Several factors contribute. First, the dataset is relatively small: 74 races with 22 drivers each yields a training matrix of around 1000 rows. Deep learning architectures generally benefit from larger datasets, and TabNet is no exception. Second, the relevant patterns in this problem are largely monotone: qualifying gap and grid position have approximately linear effects on win probability, and gradient boosting handles such patterns efficiently without requiring deep compositional representations. Third, recency weighting is implemented differently for the two models (direct sample weights for XGBoost, oversampling for TabNet), which may create subtle differences in how each model absorbs recent information.

The TabNet results are not a failure of the architecture per se: 70% accuracy is far above a reasonable baseline, and TabNet is consistent in direction (it never predicted a winner that XGBoost did not also favour, except in races where XGBoost was wrong too). Its value in this project lies more in the cross-model validation role it plays. When it agrees with XGBoost, confidence in the prediction is higher than in standalone accuracy.

6.6.2 Sensitivity to competitive context

The most striking result in Table 6.2 is TabNet’s collapse to 54.2% in 2024. This season, multiple top-tier drivers started from grid positions 1–4 in different rounds, which means the qualifying gap feature could not consistently distinguish the eventual winner. Under such conditions, TabNet’s attentive routing may select less informative features (circuit history, form, practice gaps) that increase variance without improving accuracy.

XGBoost’s resilience in 2024 (87.5%) suggests its split-based importance weighting suppresses noisy features more aggressively when the signal is weak. This is consistent with how boosting builds ensembles: early trees focus on the easiest signal, and later rounds refine residuals; in noisy rounds the residuals are informative about which features to discount.

6.6.3 Limitations

Three limitations bound the interpretation of these results. First, the operational test set is four races, too small for definitive statistical conclusions. Perfect accuracy on four rounds can happen by chance even with a mediocre model, and a single upset race in 2026 would drop XGBoost to 75%. This small sample is partly structural because two expected races had to be cancelled due to armed-conflict disruptions. Second, the 2023 and 2025 seasons were anomalously dominated by one driver, inflating historical evaluation accuracy. The realistic operating regime for this system is a competitive season like 2024, where 87.5% is a more representative figure. Third, calibration evidence still has limited depth: aggregate Brier/MAE trends are available for four races, but full driver-level probability snapshots are currently available for only two (R03 and R04).

Despite these constraints, the results support the core hypothesis: a gradient boosting model trained on qualifying-derived features and calibrated with temperature scaling can predict Formula 1 race winners with high and consistent accuracy across multiple seasons and competitive contexts.

7 Conclusions

This work set out to build an end-to-end system for Formula 1 winner prediction, from data collection and feature engineering to model training, calibrated inference, and cloud deployment. The final system is operational and reproducible: it trains offline on historical data, serves predictions through AWS Lambda, persists outputs in S3, and exposes results in Google Sheets and Looker Studio.

The main empirical conclusion is clear. XGBoost is the most reliable model for this tabular task. On the full evaluation window (2023–2026 R04), it reaches 95.9% winner accuracy (71/74), while TabNet reaches 70.3% (52/74). The historical part (2023–2025) is useful to understand behaviour across different competitive contexts, but the strongest evidence comes from unseen 2026 races in production. In the current live test window (R01–R04), XGBoost is correct in 4/4 races and TabNet in 1/4.

These results should be interpreted with appropriate caution. By calendar, six rounds were expected at this point in 2026, but two were not held because of armed-conflict disruptions. As a consequence, the production sample is still small. In addition, some historical seasons are structurally easier (for example, years with one dominant driver), which can inflate retrospective accuracy. Even with these caveats, the consistency of the model ranking across metrics (accuracy, agreement asymmetry, Brier, and MAE) supports the same conclusion: boosting generalises better than the deep tabular alternative under the current data regime.

From an engineering perspective, the project also confirms the value of the architecture itself. The pipeline produces auditable artefacts (feature importance, per-race probabilities, snapshot files), supports continuous monitoring through dashboards, and enables post-race evaluation and instrumentation. The 2026 races 1–4 constitute a frozen test set, evaluated without retraining the models: predictions were locked at the time of qualifying, and outcomes audited after each race. This approach preserves test set purity and prevents look-ahead bias.

Future work can leverage the retraining capability built into the pipeline. The architecture supports incremental retraining on newly completed races, should the project transition from this retrospective evaluation mode to a production forecasting mode where continuous adaptation is desired. The infrastructure is ready; the choice to retrain or hold frozen is a governance decision, not a technical limitation.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th acm sigkdd international conference on knowledge discovery and data mining* (pp. 2623–2631). ACM. Retrieved from <https://arxiv.org/abs/1907.10902> doi: 10.1145/3292500.3330701

- Arik, S. Ö., & Pfister, T. (2021). TabNet: Attentive interpretable tabular learning. In *Proceedings of the aaai conference on artificial intelligence* (Vol. 35, pp. 6679–6687). AAAI Press. Retrieved from <https://arxiv.org/abs/1908.07442>
- Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1–3. doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2
- Bunker, R. P., & Thabtah, F. (2019). A machine learning framework for sport result prediction. *Applied Computing and Informatics*, 15(1), 27–33. doi: 10.1016/j.aci.2017.09.005
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (pp. 785–794). ACM. Retrieved from <https://arxiv.org/abs/1603.02754> doi: 10.1145/2939672.2939785
- El Haber, E., Sawaya, E., & Attieh, M. (2025). Formula 1 race winner prediction using random forest and SHAP analysis. In *2025 international conference on engineering & mis (icemis)*. IEEE. Retrieved from <https://ieeexplore.ieee.org/abstract/document/10932140/> doi: 10.1109/ICEMIS63706.2025.10932140
- Garcia Tejada, L. (2023). *Applying machine learning to forecast formula 1 race outcomes* (Masters thesis, Aalto University). Retrieved from <https://aaltodoc.aalto.fi/items/5848c100-478d-45dd-b2e8-5caf3a3114fb>
- Gneiting, T., & Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477), 359–378. doi: 10.1198/016214506000001437
- Gorishniy, Y., Rubachev, I., Khrulkov, V., & Babenko, A. (2021). Revisiting deep learning models for tabular data. In *Advances in neural information processing systems* (Vol. 34). Curran Associates, Inc. Retrieved from <https://arxiv.org/abs/2106.11959>
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th international conference on machine learning* (Vol. 70, pp. 1321–1330). PMLR. Retrieved from <https://arxiv.org/abs/1706.04599>
- Heilmeyer, A., Graf, M., Betz, J., & Lienkamp, M. (2020). Application of monte carlo methods to consider probabilistic effects in a race simulation for circuit motorsport. *Applied Sciences*, 10(12), 4229. doi: 10.3390/app10124229
- Horvat, T., & Job, J. (2020). The use of machine learning in sport outcome prediction: A review. *WIREs Data Mining and Knowledge Discovery*, 10(5), e1380. doi: 10.1002/widm.1380
- OHanlon, E. (2023). *Using supervised machine learning to predict the final rankings of the 2021 formula one championship* (Masters thesis, National College of Ireland). Retrieved from <https://norma.ncirl.ie/6628/>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems* (Vol. 32). Curran Associates, Inc. Retrieved from <https://arxiv.org/abs/1912.01703>
- Schaffarczyk, P. (2025). *FastF1: A python package for accessing and analysing formula 1 data*. Python library. Retrieved from <https://github.com/the0ehrly/Fast-F1> (Accessed: April 2026)
- Sicoie, H. (2022). *Machine learning framework for formula 1 race winner and championship*

- standings predictor* (Bachelors thesis, Tilburg University, School of Humanities and Digital Sciences). Retrieved from <http://arno.uvt.nl/show.cgi?fid=157635>
- Silva Filho, T., Song, H., Perello-Nieto, M., Santos-Rodriguez, R., Kull, M., & Flach, P. (2023). Classifier calibration: A survey on how to assess and improve predicted class probabilities. *Machine Learning*, 112(9), 3211–3260. doi: 10.1007/s10994-023-06336-7
- Urdhwareshe, A. (2025). The use of machine learning in predicting formula 1 race outcomes. *Preprints*. Retrieved from <https://www.preprints.org/manuscript/202503.0214/v1>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems* (Vol. 30). Curran Associates, Inc. Retrieved from <https://arxiv.org/abs/1706.03762>
- Walsh, C., & Joshi, A. (2024). Machine learning for sports betting: Should model selection be based on accuracy or calibration? *Machine Learning with Applications*, 16, 100539. doi: 10.1016/j.mlwa.2024.100539
- Weissbock, J., & Mills, S. (2025). *Evaluating the predictive power of qualifying performance in formula one grand prix*. arXiv preprint arXiv:2507.10966. Retrieved from <https://arxiv.org/abs/2507.10966>
-

List of Acronyms and Abbreviations

AI	Artificial Intelligence.
API	Application Programming Interface.
Athena	Amazon Athena.
AUC	Area Under the Curve.
AWS	Amazon Web Services.
BS	Brier Score.
CPU	Central Processing Unit.
CSV	Comma-Separated Values.
CV	Cross-Validation.
DL	Deep Learning.
DNF	Did Not Finish.
DRS	Drag Reduction System.
ECR	Elastic Container Registry.
F1	Formula 1.
FP1	Free Practice 1.
FP2	Free Practice 2.
FP3	Free Practice 3.
GBDT	Gradient-Boosted Decision Tree.
GPU	Graphics Processing Unit.
HP	Hyperparameter.
JSON	JavaScript Object Notation.
Lambda	AWS Lambda.
MAE	Mean Absolute Error.
ML	Machine Learning.
MLOps	Machine Learning Operations.
MLP	Multilayer Perceptron.
NLL	Negative Log-Likelihood.
NN	Neural Network.
RF	Random Forest.
ROC-AUC	Receiver Operating Characteristic Area Under the Curve.
S3	Simple Storage Service.
SC	Safety Car.
SHAP	SHapley Additive exPlanations.

SQL	Structured Query Language.
SVM	Support Vector Machine.
TFG	Trabajo Final de Grado.
TPE	Tree-structured Parzen Estimator.
VSC	Virtual Safety Car.
